

Agent Engineer Take-Home

Objective

We're giving you a working virtual blackjack casino as a starter repo - tables, a few player and dealer strategies, and a stub that logs round outcomes. It's ordinary code, and the casino itself is not the point. Your job is to treat it as a real, evolving codebase and build a small layer of agents that maintain it the way you'd want a production repo maintained: reviewing changes, writing tests, updating docs, keeping dependencies current. The goal is to show how you design agents that do real software-development work on their own initiative.

Getting Started

The starter repo is at <https://github.com/OpenHands/casino-skeleton>

Click "Use this template" to create your own copy - this preserves the existing commit history, which we'll want to see alongside your agents' commits. Please make your repo private and add your interviewers as collaborators.

It's Python 3, no setup beyond pytest:

- `pip install pytest`
- `python3 -m pytest tests/ -q`
- `python3 -m casino.simulate`

Requirements

- Build at least 3 agents that proactively modify the repo - real SDLC work such as code review, test writing, documentation, or dependency updates - without you prompting each action during the recording.
- Each agent needs a trigger it responds to on its own - a commit, a timer, a failing test, an anomaly in the casino's output - rather than a manual go-ahead from you.
- Leave the layer running unattended for a stretch of the recording so we can see it act on its own.

These don't need to be sophisticated. An agent that makes one LLM call to write a missing test and commit it counts, as long as it fired on its own and did something real.

Beyond that, you are free to choose how to implement it. You could:

- Trigger agents off git hooks, a polling loop, a CI run, or something else entirely
- Have the agents work independently, or coordinate through a queue or orchestrator
- Build out monitoring or a dashboard, if you want richer signals for your agents to react to
- Extend the casino itself, if you want to give your agents more to chew on - though we're not evaluating the casino

Show us what you do best. We are not looking for one specific outcome. We want to see how you decide what's worth automating in a codebase, how you wire up agents that act without you in the loop, and how you scope that under limited time.

A note on using AI

You're welcome to use AI tools (Copilot, OpenHands, GPT, etc.) to help you build this - as with any exercise, you should be directing the solution and reviewing what gets committed. That's separate from the agents you're

building: those are meant to act autonomously as part of the deliverable, not something you hand-hold through during the recording. We want to see you build the delegation, not just use a delegate.

Time Limit

Spend no more than 2 hours total, including setup. Done is better than perfect. We care more about your reasoning and prioritization than polish.

What We Are Looking For

- How you decide what's worth automating in a codebase
- How quickly and cleanly you can stand up working agents
- How transparent the system is to someone who wasn't watching it run
- How production-ready your instincts are
- Whether the agents do real work, triggered by something other than you

What to Submit

- A link to your screen recording
- A GitHub or GitLab repo with your code, with the agents' commits in the history
- A short README (1 to 2 paragraphs) that includes:
 - Instructions for running the agent layer
 - What each agent does and what triggers it
 - What AI tools you used and how
 - Anything that did not go as planned or you would improve with more time